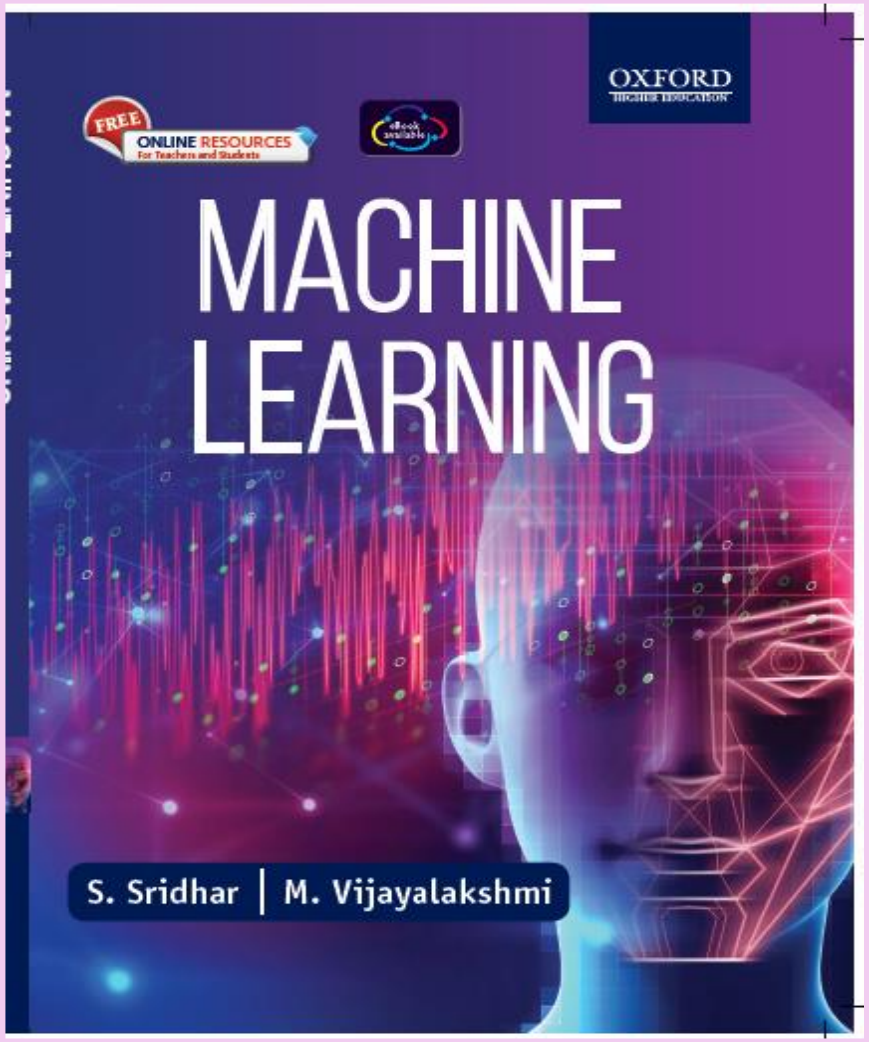


OXFORD
UNIVERSITY PRESS

Machine Learning

S. Sridhar and M. Vijayalakshmi

© Oxford University Press 2021. All rights reserved

Chapter 10

Artificial Neural Network

© Oxford University Press 2021. All rights reserved

Artificial Neural Networks

Artificial Neural Networks (ANNs) imitate human brain behaviour and the way in which learning happens in a human. The human brain constitutes a mass of neurons that are all connected as a network, which is actually a directed graph. These neurons are the processing units which receive information, process it and then transmit this data to other neurons that allows humans to learn almost any task. ANN is a learning mechanism that models a human brain to solve any non-linear and complex problem. Each neuron is modelled as a computing unit, or simply called as a node in ANN, that is capable of doing complex calculations. ANN is a system that consists of many such computing units operating in parallel that can learn from observations. They are widely used in developing artificial learning systems and have inspired researchers and industry in Machine Learning nowadays. Some typical applications of ANN in the field of computer science are Natural Language Processing (NLP), pattern recognition, face recognition, speech recognition, character recognition, text processing, stock prediction, computer vision, etc. ANNs also have been considerably used in other engineering fields such as Chemical industry, Medicine, Robotics, Communications, Banking, and Marketing. This chapter aims to introduce the concepts necessary to understand the working of Artificial Neural Networks.

© Oxford University Press 2021. All rights reserved

Introduction

The human nervous system has billions of neurons that are the processing units which make humans to perceive things, to hear, to see and to smell. The human nervous system works beautifully, making us understand who we are, what we do, where we are and everything in our surrounding. It makes us to remember, recognize and correlate things around us. It is a learning system that consists of functional units called nerve cells, typically called as neurons. The human nervous system is divided into two sections called the Central Nervous System (CNS) and the Peripheral Nervous System (PNS). The brain and the spinal cord constitute the CNS and the neurons inside and outside the CNS constitute the PNS. The neurons are basically classified into three types called sensory neurons, motor neurons and interneurons. Sensory neurons get information from different parts of the body and bring it into the CNS, whereas motor neurons receive information from other neurons and transmit commands to the body parts. The CNS consists of only interneurons which connect one neuron to another neuron by receiving information from one neuron and transmitting it to another. The basic functionality of a neuron is to receive information, process it and then transmit it to another neuron or to a body part.

© Oxford University Press 2021. All rights reserved

Biological Neurons

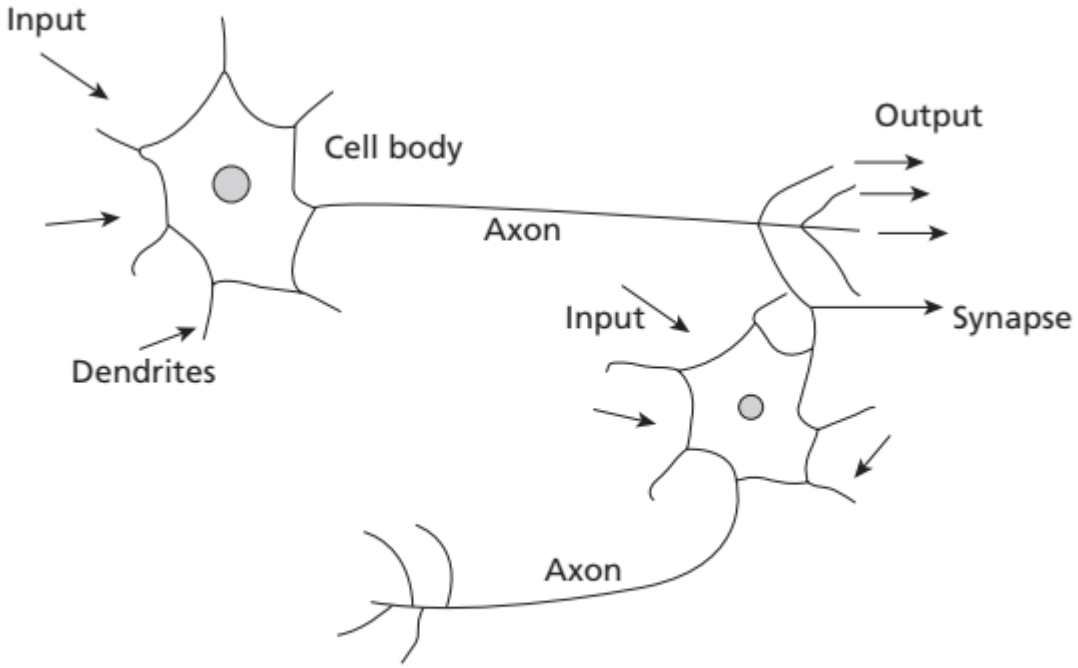


Figure 10.1: A Biological Neuron

© Oxford University Press 2021. All rights reserved

Artificial Neurons

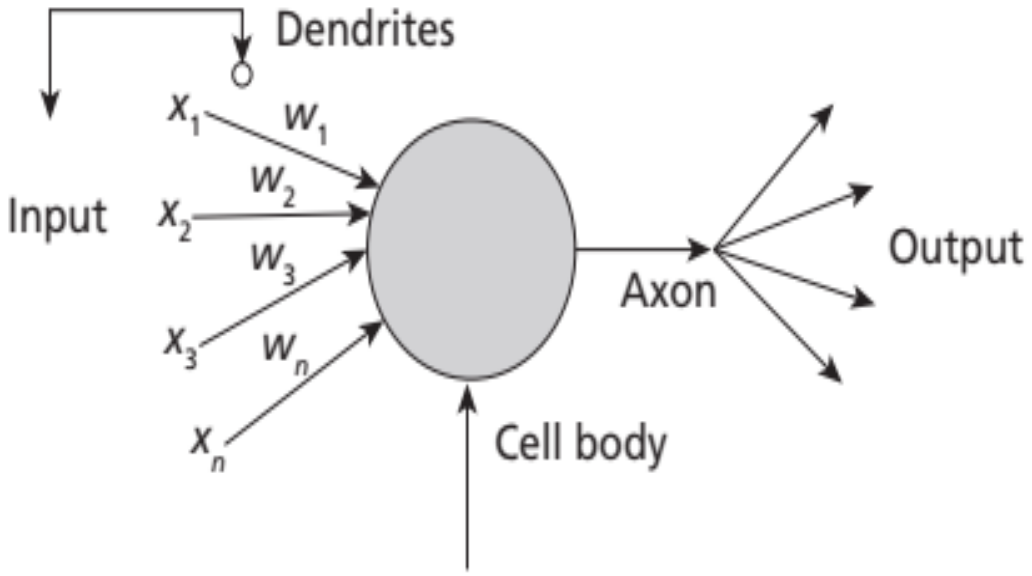


Figure 10.2: An Artificial Neuron

© Oxford University Press 2021. All rights reserved

Simple Model of an Artificial Neuron

The first mathematical model of a biological neuron was designed by McCulloch & Pitts in 1943. It includes two steps:

1. It receives weighted inputs from other neurons
2. It operates with a threshold function or activation function

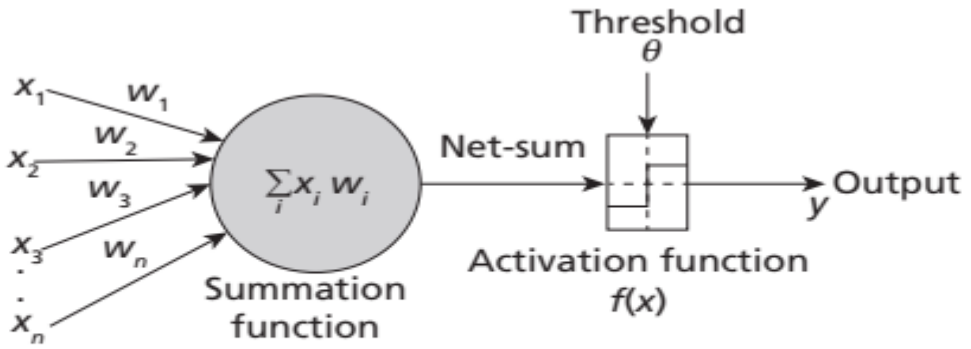


Figure 10.3: McCulloch & Pitts Neuron Mathematical Model

© Oxford University Press 2021. All rights reserved

Artificial Neural Network Structure

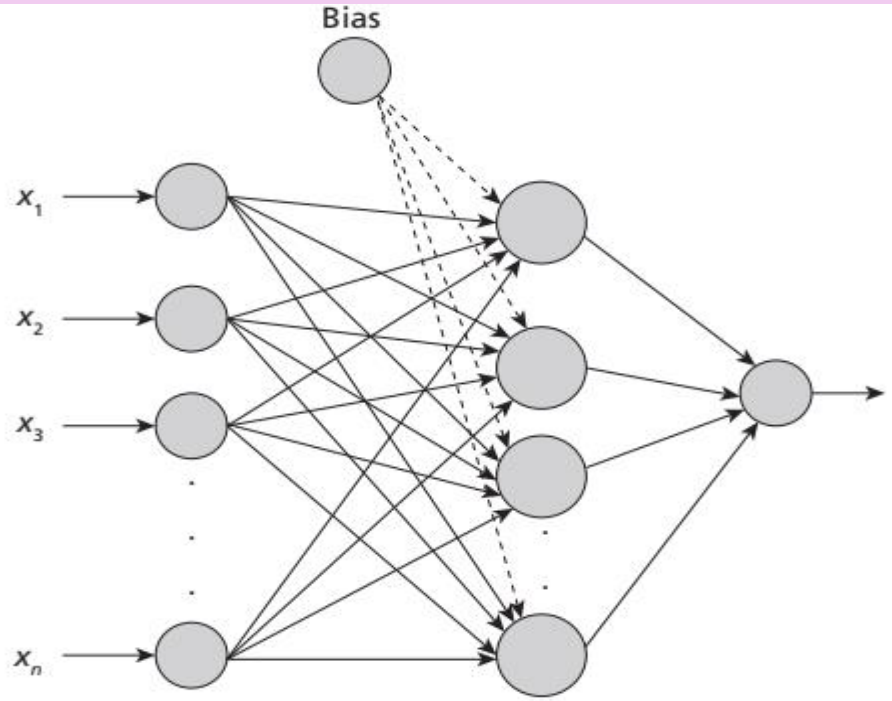


Figure 10.4: Artificial Neural Network Structure

© Oxford University Press 2021. All rights reserved

Activation Functions

Activation functions are mathematical functions associated with each neuron in the neural network that map input signals to output signals. It decides whether to fire a neuron or not based on the input signals the neuron receives. These functions normalize the output value of each neuron either between 0 and 1 or between −1 and +1. Typical activation functions can be linear or non-linear.

© Oxford University Press 2021. All rights reserved

Activation Functions

1. Identity Function or Linear Function

$$f(x) = x \ \forall x \tag{10.4}$$

The value of $f(x)$ increases linearly or proportionally with the value of x. This function is useful when we do not want to apply any threshold. The output would be just the weighted sum of input values. The output value ranges between $-\infty$ and $+\infty$.

2. Binary Step Function

$$f(x) = \begin{cases} 1 & if\ f(x) \geq \theta \\ 0 & if\ f(x) < \theta \end{cases} \tag{10.5}$$

The output value is binary, i.e., 0 or 1 based on the threshold value θ. If value of $f(x)$ is greater than or equal to θ, it outputs 1 or else it outputs 0.

3. Bipolar Step Function

$$f(x) = \begin{cases} 1 & if\ f(x) \geq \theta \\ -1 & if\ f(x) < \theta \end{cases} \tag{10.6}$$

The output value is bipolar, i.e., +1 or −1 based on the threshold value θ. If value of $f(x)$ is greater than or equal to θ, it outputs +1 or else it outputs −1.

© Oxford University Press 2021. All rights reserved

Activation Functions

4. Sigmoidal Function or Logistic Function

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{10.7}$$

It is a widely used non-linear activation function which produces an *S*-shaped curve and the output values are in the range of 0 and 1. It has a vanishing gradient problem, i.e., no change in the prediction for very low input values and very high input values.

5. Bipolar Sigmoid Function

$$\sigma(x) = \frac{1 - e^{-x}}{1 + e^{-x}} \tag{10.8}$$

It outputs values between –1 and +1.

6. Ramp Functions

$$f(x) = \begin{cases} 1 & if\ x > 1 \\ x & if\ 0 \leq x \leq 1 \\ 0 & if\ x < 0 \end{cases} \tag{10.9}$$

It is a linear function whose upper and lower limits are fixed.

© Oxford University Press 2021. All rights reserved

Activation Functions

7. Tanh – Hyperbolic Tangent Function

The Tanh function is a scaled version of the sigmoid function which is also non-linear. It also suffers from the vanishing gradient problem. The output values range between –1 and 1.

$$\tan h(x) = \frac{2}{1 + e^{-2x}} - 1 \tag{10.10}$$

8. ReLu – Rectified Linear Unit Function

This activation function is a typical function generally used in deep learning neural network models in the hidden layers. It avoids or reduces the vanishing gradient problem. This function outputs a value of 0 for negative input values and works like a linear function if the input values are positive.

$$r(x) = \max(0, x) = \begin{cases} x & if\ x \geq 0 \\ 0 & if\ x < 0 \end{cases} \tag{10.11}$$

9. Softmax Function

This is a non-linear function used in the output layer that can handle multiple classes. It calculates the probability of each target class which ranges between 0 and 1. The probability of the input belonging to a particular class is computed by dividing the exponential of the given input value by the sum of the exponential values of all the inputs.

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=0}^{k} e^{x_j}} \text{ where } i = 0 \ldots k \tag{10.12}$$

© Oxford University Press 2021. All rights reserved

Perceptron and Learning Theory

THE PERCEPTRON MODEL CONSISTS OF 4 STEPS:
1. INPUTS FROM OTHER NEURONS
2. WEIGHTS AND BIAS
3. NET SUM
4. ACTIVATION FUNCTION

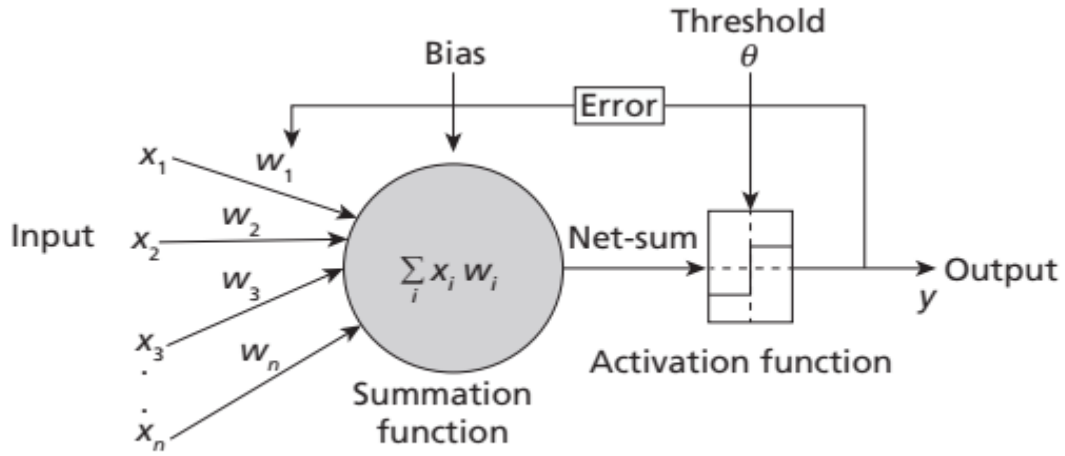


Figure 10.5: Perceptron Model

© Oxford University Press 2021. All rights reserved

Perceptron Algorithm

Algorithm 10.1: Perceptron Algorithm

Set initial weights $w_1, w_2, \ldots, w_n$ and bias θ to a random value in the range [–0.5, 0.5].

For each Epoch,

1. Compute the weighted sum by multiplying the inputs with the weights and add the products.
2. Apply the activation function on the weighted sum:

$$Y = \text{Step}\ ((x_1 w_1 + x_2 w_2) - \theta)$$

3. If the sum is above the threshold value, output the value as positive else output the value as negative.
4. Calculate the error by subtracting the estimated output $Y_{estimated}$ from the desired output $Y_{desired}$:

$$\text{error } e(t) = Y_{desired} - Y_{estimated}$$

[If error $e(t)$ is positive, increase the perceptron output Y and if it is negative, decrease the perceptron output Y.]

5. Update the weights if there is an error:

$$\Delta w_i = \propto \times e(t) \times x_i$$

$$w_i = w_i + \Delta w_i$$

where, x_i is the input value, $e(t)$ is the error at step t, $\propto$ is the learning rate and Δw_i is the difference in weight that has to be added to w_i.

© Oxford University Press 2021. All rights reserved

XOR Problem

A PERCEPTRON MODEL CAN SOLVE ALL BOOLEAN FUNCTIONS WHICH ARE LINEARLY SEPARABLE. HOWEVER, THE XOR PROBLEM WAS IDENTIFIED IN 1969 BY MINSKY AND PAPERT.

SINCE XOR IS NOT LINEARLY SEPARABLE, THE SINGLE LAYER PERCEPTRON FAILED TO CLASSIFY AND HENCE THIS PROBLEM LED TO THE EVOLUTION OF A MULTI-LAYER PERCEPTRON.

INITIALLY, THE MULTI-LAYER PERCEPTRON (MLP) WAS NOT A SUCCESS DUE TO THE LACK OF AN APPROPRIATE LEARNING ALGORITHM.

IN 1974, WERBOS INTRODUCED A BACK PROPAGATION ALGORITHM FOR THE THREE-LAYERED PERCEPTRON NETWORK, AND IN 1986, A GENERAL BACK PROPAGATION ALGORITHM FOR A MULTI-LAYERED PERCEPTRON WAS INTRODUCED BY RUMMELHART AND MCLELLAND.

THEN, AGAIN, ANN AND DEEP NEURAL NETWORKS BECAME A SUCCESS, SOLVING MANY COMPLEX PROBLEMS IN THE CURRENT ERA.

© Oxford University Press 2021. All rights reserved

Delta Learning Rule and Gradient Descent

Generally, learning in neural networks is performed by adjusting the network weights in order to minimize the difference between the desired and estimated outputs. This delta difference is measured as an error function or also called as cost function. The cost function, being linear and continuous, is differentiable. This way of learning called as delta rule (also known as Widrow-Hoff" rule or Adaline rule) is a type of back propagation applied for training the network. The training error of a hypothesis is half the squared difference between the desired target output and actual output and is given as follows:

$$\text{Training Error} = \frac{1}{2}\sum_{d \in T}(O_{Desired} - O_{Estimated})^2 \quad (10.16)$$

where, T is the training dataset, $O_{Desired}$ and $O_{Estimated}$ are the desired target output and estimated actual output, respectively, for a training instance d.

The principle of gradient descent is an optimization approach which is used to minimize the cost function by converging to a local minimal point moving in the negative direction of the gradient and each step size during movement is determined by the learning rate and the slope of the gradient.

© Oxford University Press 2021. All rights reserved

Types of Artificial Neural Networks

FEED FORWARD NEURAL NETWORK

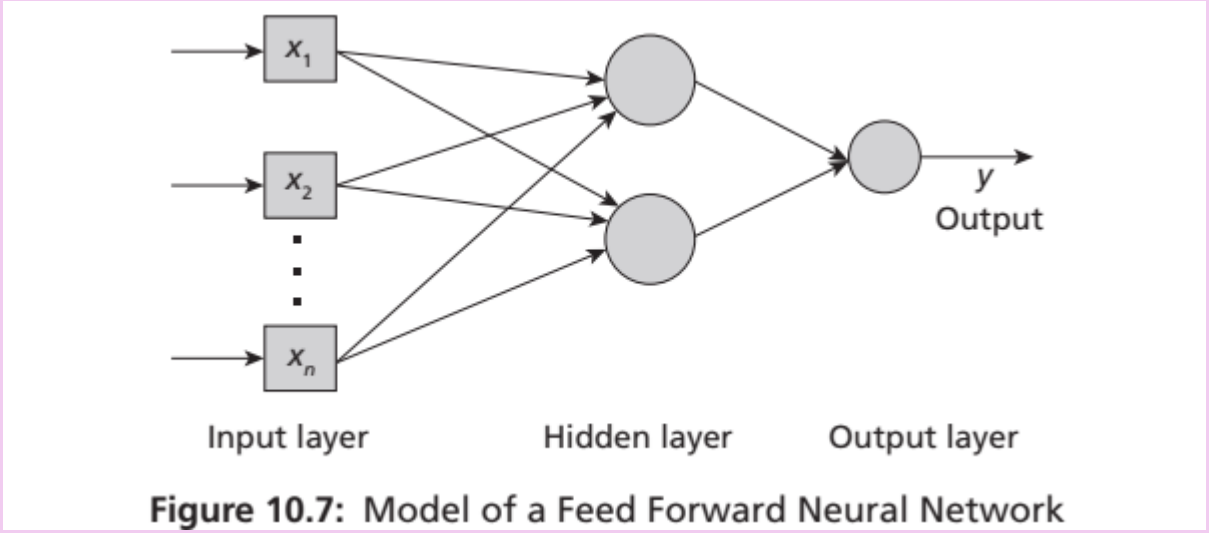


Figure 10.7: Model of a Feed Forward Neural Network

© Oxford University Press 2021. All rights reserved

Types of Artificial Neural Networks

FULLY CONNECTED NEURAL NETWORK

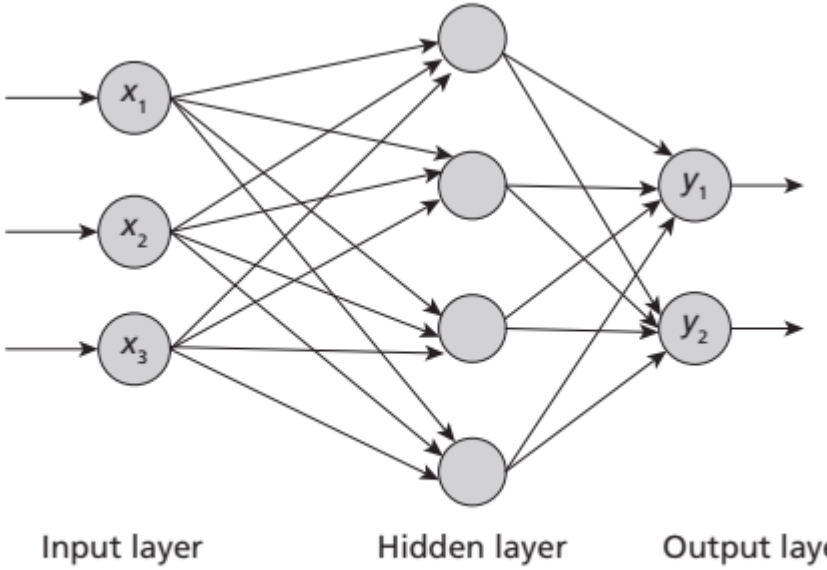


Figure 10.8: Model of a Fully Connected Neural Network

© Oxford University Press 2021. All rights reserved

Types of Artificial Neural Networks

MULTI-LAYER PERCEPTRON (MLP)

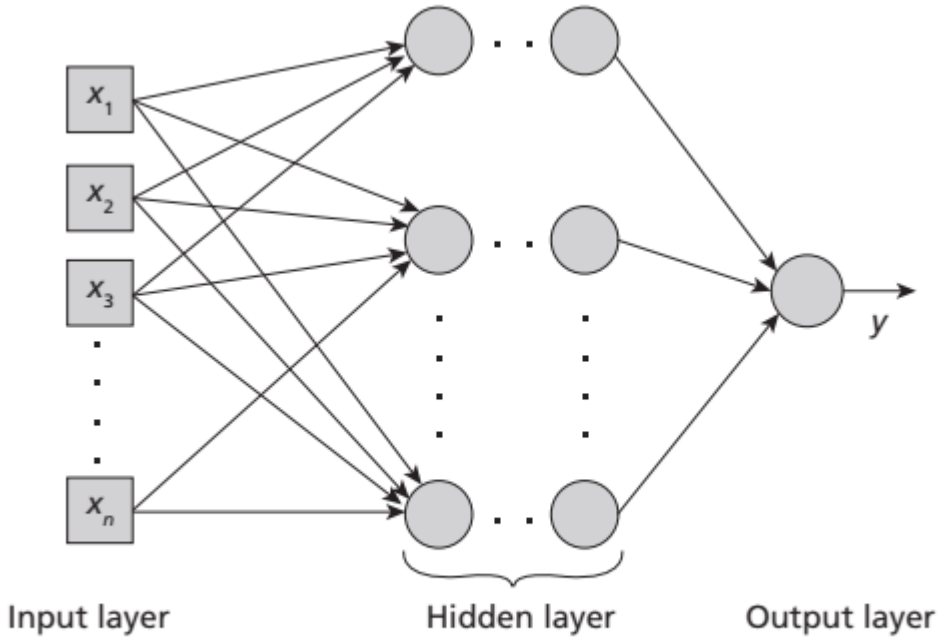


Figure 10.9: Model of a Multi-Layer Perceptron

© Oxford University Press 2021. All rights reserved

Types of Artificial Neural Networks

FEEDBACK NEURAL NETWORK

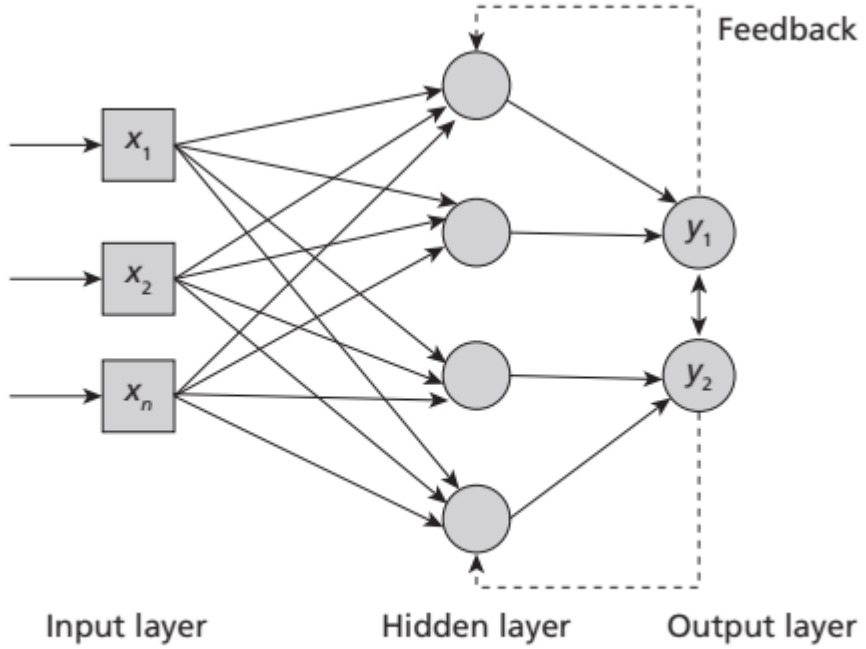


Figure 10.10: Model of a Feedback Neural Network

© Oxford University Press 2021. All rights reserved

Learning in a Multi-layer Perceptron

Algorithm 10.2: Learning in an MLP

Input: Input vector $(x_1, x_2, ..., x_n)$

Output: Y_n

Learning rate: α

Assign random weights and biases for every connection in the network in the range [–0.5, +0.5].

Step 1: Forward Propagation

1. **Calculate Input and Output in the *Input Layer*:**

   (Input layer is a direct transfer function, where the output of the node equals the input).

   Input at Node j 'I$_j$' in the Input Layer is

$$I_j = x_j$$

© Oxford University Press 2021. All rights reserved

Learning in a Multi-layer Perceptron

where,

x_j is the input received at Node j

Output at Node j 'O_j' in the Input Layer is

$$O_j = I_j$$

2. **Calculate Net Input and Output in the *Hidden Layer* and *Output Layer*:**

*Net Input at Node j in the **Hidden Layer** is*

$$I_i = \sum_{i=1}^{n} x_i w_{ij} + x_0 \times \theta_j$$

where,

x_i is the input from Node i

w_{ij} is the weight in the link from Node i to Node j

x_0 is the input to bias node '0' which is always assumed as 1

θ_j is the weight in the link from the bias node '0' to Node j

*Net Input at Node j in the **Output Layer** is*

$$I_j = \sum_{i=1}^{n} O_i w_{ij} + x_0 \times \theta_j$$

where,

O_i is the output from Node i

w_{ij} is the weight in the link from Node i to Node j

x_0 is the input to bias node '0' which is always assumed as 1

θ_j is the weight in the link from the bias node '0' to Node j

Output at Node j

$$O_j = \frac{1}{1 + e^{-I_j}}$$

where,

I_j is the input received at Node j

© Oxford University Press 2021. All rights reserved

Learning in a Multi-layer Perceptron

3. **Estimate error at the node in the *Output Layer*:**

$$\text{Error} = O_{Desired} - O_{Estimated}$$

where,

$O_{Desired}$ is the desired output value of the Node in the Output Layer

$O_{Estimated}$ is the estimated output value of the Node in the Output Layer

Step 2: Backward Propagation

1. **Calculate Error at each node:**

For each Unit k in the ***Output Layer***

$$\text{Error}_k = O_k(1 - O_k)\,(O_{Desired} - O_k)$$

where,

O_k is the output value at Node k in the Output Layer.

$O_{Desired}$ is the desired output value of the Node in the Output Layer.

For each unit j in the ***Hidden Layer***

$$\text{Error}_j = O_j\,(1 - O_j)\ \Sigma_k\,\text{Error}_k\ w_{jk}$$

where,

O_j is the output value at Node j in the Hidden Layer.

Error_k is the error at Node k in the Output Layer.

w_{jk} is the weight in the link from Node j to Node k.

© Oxford University Press 2021. All rights reserved

Learning in a Multi-layer Perceptron

2. Update all weights and biases:

Update weights

$$\Delta w_{ij} = \propto \times \text{Error}_j \times O_i$$
$$w_{ij} = w_{ij} + \Delta w_{ij}$$

where,

O_i is the output value at Node i.

Error_j is the error at Node j.

$\propto$ is the learning rate.

w_{ij} is the weight in the link from Node i to Node j.

Δw_{ij} is the difference in weight that has to be added to w_{ij}.

Update Biases

$$\Delta \theta_j = \propto \times \text{Error}_j$$
$$\theta_j = \theta_j + \Delta \theta_j$$

where,

Error_j is the error at Node j.

$\propto$ is the learning rate.

θ_j is the bias value from Bias Node 0 to Node j.

$\Delta \theta_j$ is the difference in bias that has to be added to θ_j.

© Oxford University Press 2021. All rights reserved

Radial Basis Function Neural Network

Typical Radial Basis Functions (RBF) are:

The Gaussian RBF which monotonically decreases with distance from the centre.

$$H(x) = e^{\frac{-(x-c)^2}{r^2}} \tag{10.17}$$

where, c is the centre and r is the radius.

A Multiquadric RBF which monotonically increases with distance from the centre.

$$H(x) = \sqrt{\frac{r^2 + (x-c)^2}{r}} \tag{10.18}$$

© Oxford University Press 2021. All rights reserved

Architecture of RBFNN

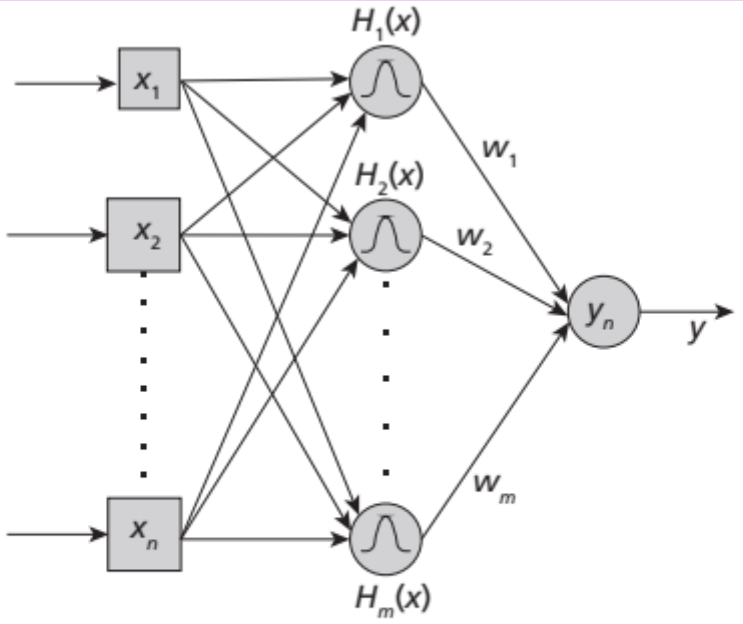


Figure 10.12: Architecture of RBFNN

© Oxford University Press 2021. All rights reserved

Algorithm 10.3: Radial Basis Function Neural Network

Algorithm 10.3: Radial Basis Function Neural Network

Input: Input vector $(x_1, x_2, ..., x_n)$

Output: Y_n

Assign random weights for every connection from the Hidden layer to the Output layer in the network in the range [–1, +1].

Forward Phase:

Step 1: Calculate Input and Output in the Input Layer:

(Input layer is a direct transfer function, where the output of the node equals the input).

Input at Node i 'I_i' in the Input Layer is

$$\boxed{I_i = x_i}$$

where,

x_i is the input received at Node i.

© Oxford University Press 2021. All rights reserved

Algorithm 10.3: Radial Basis Function Neural Network

Output at Node i 'O_i' in the Input Layer is

$$O_i = I_i$$

Step 2: For each node j in the Hidden Layer, find the centre/receptor c and the variance r.
Define hidden layer neurons with Gaussian RBF whose output is:

$$H_j(x) = e^{\frac{-(x-c_j)^2}{r^2}}$$

where, x is the input, c_j is the centre and r is the radius.
Compute $(x - c_j)^2$ applying Euclidean distance measure between x and c_j.

Step 3: For each node k in the Output Layer, compute linear weighted sum of the output of each neuron k from the hidden layer neurons j.

$$F_k(x) = \sum_{j=1}^{m} w_{jk} H_j(x)$$

where,
w_{jk} is the weight in the link from the Hidden Layer neuron j to the Output Layer neuron k.
$H_j(x)$ is the output of a Hidden Layer neuron j for an input vector x.

Backward Phase:

Step 1: Train the Hidden layer using Back propagation.

Step 2: Update the weights between the Hidden layer and Output layer.

© Oxford University Press 2021. All rights reserved

Self-organizing Feature Map

Self-Organizing Feature Map (SOFM) is a special type of Feed Forward Artificial Neural Network developed by Dr Teuvo Kohonen in 1982. Kohonen network is a competitive learning network or also called as adaptive learning network. SOFM is an unsupervised learning model that clusters data by mapping a high-dimensional data into a two-dimensional map (neurons) or plane. The model learns to cluster or self organize a high-dimensional data without knowing the class membership of the input data, and hence the name self-organizing nodes. These self-organizing nodes are also called as feature maps. The mapping is based on the relative distance or similarity between the points and the points that are near to each other in the input space are mapped to nearby output map units in the SOFM.

© Oxford University Press 2021. All rights reserved

Network Architecture of Self Organizing Feature Map

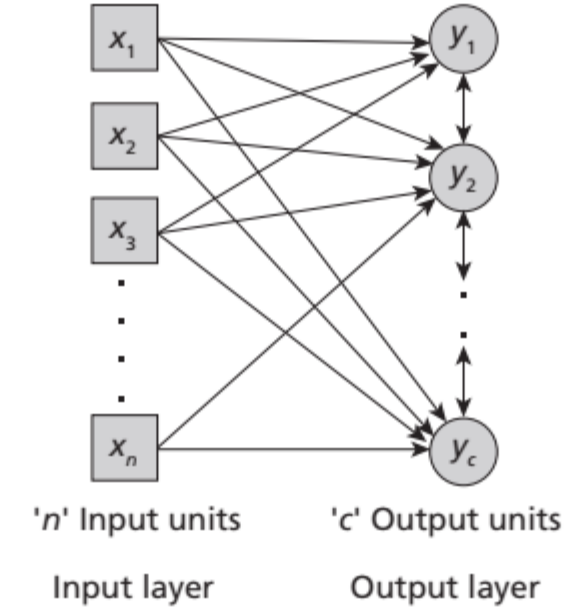


Figure 10.14: Network Architecture of Self Organizing Feature Map

© Oxford University Press 2021. All rights reserved

Algorithm 10.4: Self-Organizing Feature Map

Algorithm 10.4: Self-Organizing Feature Map

Input: 'm' Training Vectors (x_1, x_2, ..., x_m), each of length 'n':

$$\begin{bmatrix} x_{1,1} & x_{1,2} \ldots . x_{1,n} \\ x_{2,1} & x_{2,2} \ldots . . x_{2,n} \\ & \\ & \\ x_{m,1} & x_{m,2} \ldots . . x_{m,n} \end{bmatrix}$$

Output: 'c' categories

Input Layer: 'n' Input units (Length of the training vector)

Output Layer: 'c' Output units (Number of categories)

Step 1: Initialize random weights w_j between 0 and 1, for each Output unit j.
(A Weight vector of length 'n' is associated with each Output unit)

Step 2: For each training sample x_s:

(a) Compute Euclidean distance score between the training sample x_s and the weight vector w_j of each output unit:

© Oxford University Press 2021. All rights reserved

Algorithm 10.4: Self-Organizing Feature Map

$$d^2 = (\text{Euclidean distance})^2 = \sum_{k=1}^{n} (x_{s,k} - w_{j,k}(t))^2$$

(b) Choose the winning output unit which has smaller distance to the input sample as the best matching unit.

(c) Update the weights for the winning unit:

$$w_j(t+1) = w_j(t) + \eta(t)(x_s - w_j(t))$$

where,

$\eta(t)$ is the Learning rate.

$w_j(t)$ is the weight of the winning unit at step t.

$w_j(t+1)$ is the updated weight of the winning unit at step $(t+1)$.

Step 3: Repeat until the feature map doesn't change.

© Oxford University Press 2021. All rights reserved

Popular Applications of Artificial Neural Networks

ANN learning mechanisms are used in many complex applications that involve modelling of non-linear processes. ANN is a useful model that can handle even noisy and incomplete data. They are used to model complex patterns, recognize patterns and solve prediction problems like humans in many areas such as:

1. Real-time applications: Face recognition, emotion detection, self-driving cars, navigation systems, routing systems, target tracking, vehicle scheduling, etc.
2. Business applications: Stock trading, sales forecasting, customer behaviour modelling, Market research and analysis, etc.
3. Banking and Finance: Credit and loan forecasting, fraud and risk evaluation, currency price prediction, real-estate appraisal, etc.
4. Education: Adaptive learning software, student performance modelling, etc.
5. Healthcare: Medical diagnosis or mapping symptoms to a medical case, image interpretation and pattern recognition, drug discovery, etc.
6. Other Engineering Applications: Robotics, aerospace, electronics, manufacturing, communications, chemical analysis, food research, etc.

© Oxford University Press 2021. All rights reserved

Advantages and Disadvantages of ANN

ADVANTAGES OF ANN

1. ANN can solve complex problems involving non-linear processes.
2. ANNs can learn and recognize complex patterns and solve problems as humans solve a problem.
3. ANNs have a parallel processing capability and can predict in less time.
4. They have an ability to work with inadequate knowledge. It can even handle incomplete and noisy data.
5. They can scale well to larger data sets and outperforms other learning mechanisms.

© Oxford University Press 2021. All rights reserved

Advantages and Disadvantages of ANN

LIMITATIONS OF ANN

1. An ANN requires processors with parallel processing capability to train the network running for many epochs. The function of each node requires a CPU capability which is difficult for very large networks with a large amount of data.
2. They work like a 'black box' and it is exceedingly difficult to understand their working in inner layers. Moreover, it is hard to understand the relationship between the representations learned at each layer.

3. The modelling with ANN is also extremely complicated and the development takes a much longer time.
4. Generally, neural networks require more data than traditional machine learning algorithms, and they do not perform well on small datasets.
5. They are also more computationally expensive than traditional learning techniques.

© Oxford University Press 2021. All rights reserved

Challenges of Artificial Neural Networks

The major challenges while modelling a real-time application with ANNs are:

1. Training a neural network is the most challenging part of using this technique. Overfitting or underfitting issues may arise if datasets used for training are not correct. It is also hard to generalize to the real-world data when trained with some simulated data. Moreover, neural network models normally need a lot of training data to be robust and are usable for a real-time application.
2. Finding the weight and bias parameters for neural networks is also hard and it is difficult to calculate an optimal model.

© Oxford University Press 2021. All rights reserved

Summary

1. The human nervous system has billions of neurons that are the processing units which make us to perceive things around us, to hear, to see and to smell.
2. A typical biological neuron has four parts called dendrites, soma, axon and synapse. The body of the neuron is called as soma.
3. The neurons form a network structure which processes input information and gives out a response.
4. Artificial neurons are like biological neurons which are called as nodes that can receive one or more input information from other neurons and process the information.
5. The first mathematical model of a biological neuron designed by McCulloch & Pitts receives weighted inputs from other neurons and operates with an activation function.
6. A neuron gets fired when the weighted sum of the received inputs exceeds the threshold of the activation function.
7. An Artificial Neural Network has a network structure represented as a directed graph with a set of neuron nodes and connection links or edges connecting the nodes.
8. The activation function decides whether to fire a neuron or not based on the input signals the neuron receives.
9. Activation functions can be linear or non-linear.
10. Linear functions are generally used in binary perceptrons, where the input values can be classified into any one of the two groups.
11. Non-linear functions are continuous functions that map the input in the range of (0, 1) or (−1, 1), etc.
12. The first neural network model 'Perceptron' is a linear binary classifier used for supervised learning.
13. The Perceptron, designed by Frank Rosenblatt, could actually learn weights and thresholds from training data by itself.

© Oxford University Press 2021. All rights reserved

Summary

14. A perceptron model can solve any Boolean functions which are linearly separable.
15. XOR is not linearly separable, so a single layer perceptron fails to classify, and hence this problem led to the evolution of a Multi-Layer Perceptron (MLP).
16. Different types of ANNs differ by the network structure, activation function used and the learning rules used. In an ANN, there are three layers called input layer, hidden layer and output layer.
17. An MLP has atleast one hidden layer and as the number of hidden layers is increased, the learning becomes more complex and they form a deep neural network.
18. An MLP uses back propagation for supervised learning of the network.
19. The hidden layer in the RBFNN uses a non-linear radial basis function as the activation function which converts the input parameters into a high dimension space which is then fed into the network to linearly separate the problem.
20. Self-Organizing Feature Map is an unsupervised learning model that clusters data by mapping a high-dimensional data into a two-dimensional map (neurons) or plane.

© Oxford University Press 2021. All rights reserved